

DISEASE MANAGEMENT

1. INTRODUCTION

The Disease Management System is a comprehensive platform that enables healthcare professionals to efficiently manage patient data, track disease information, create and implement treatment protocols, and administer diagnostic tests across different. With its robust features and user-friendly interface, the system streamlines the process of disease management and helps healthcare providers deliver high-quality care to their patients.

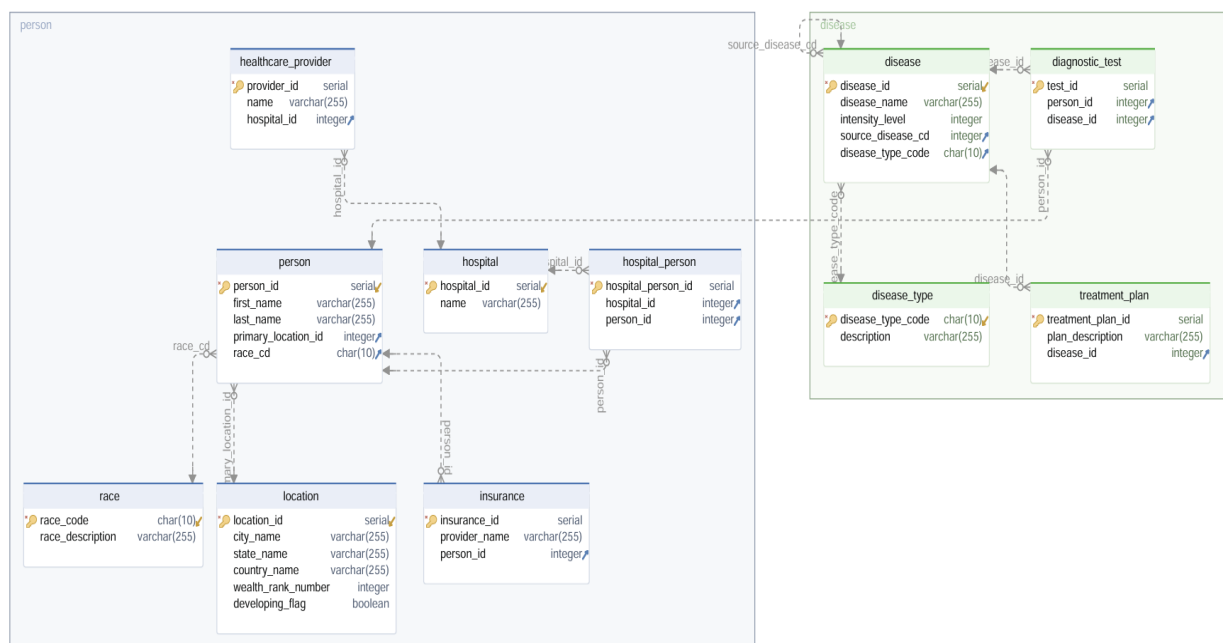
2. BUSINESS PROBLEM

For a Disease Model, a typical business problem: "Enhancing disease management for healthcare providers by offering a comprehensive system that facilitates efficient tracking, diagnosis, and treatment of diseases, ultimately aiming to improve patient outcomes and streamline research and data analysis processes."

3. DATABASE DESIGN:

To efficiently manage the healthcare related information, a relational database schema has been designed with tables that are specifically tailored to store data related to diseases, treatment plans, diagnostic tests, and healthcare entities. This schema is aimed at providing an organized and comprehensive approach to handling healthcare data, making it easier to access and analyse important information when needed.

3.1. ER Diagram:



3.2. Data Dictionary:

Disease_Type

- **Disease_Type_Code (CHAR(10), PRIMARY KEY):** A unique code identifying a type of disease.
- **Description (VARCHAR(255)):** A textual description of the disease type.

Disease

- **Disease_ID (SERIAL, PRIMARY KEY):** Unique identifier for a disease.
- **Disease_Name (VARCHAR(255)):** Name of the disease.
- **Intensity_Level (INT):** Indicates the severity or intensity of the disease.
- **Source_Disease_Cd (INT, REFERENCES Disease(Disease_ID)):** Optional reference to another disease that may be the source or related.
- **Disease_Type_Code (CHAR(10), REFERENCES Disease_Type(Disease_Type_Code)):** Code linking to the Disease_Type table for categorization.

Location

- **Location_ID (SERIAL, PRIMARY KEY):** Unique identifier for a location.
- **City_Name (VARCHAR(255)):** Name of the city.
- **State_Name (VARCHAR(255)):** Name of the state.
- **Country_Name (VARCHAR(255)):** Name of the country.
- **Wealth_Rank_Number (INT):** A ranking of the location's wealth.
- **Developing_Flag (BOOLEAN):** True if the location is in a developing region; otherwise, false.

Race

- **Race_Code (CHAR(10), PRIMARY KEY):** Unique code for identifying a race.
- **Race_Description (VARCHAR(255)):** Description of the race.

Person

- **Person_ID (SERIAL, PRIMARY KEY):** Unique identifier for an individual.
- **First_Name (VARCHAR(255)):** First name of the individual.
- **Last_Name (VARCHAR(255)):** Last name of the individual.
- **Primary_Location_ID (INT, REFERENCES Location(Location_ID)):** ID linking to the primary location of the individual.
- **Race_CD (CHAR(10), REFERENCES Race(Race_Code)):** Code linking to the Race table for demographic analysis.

Treatment_Plan

- **Treatment_Plan_ID (SERIAL, PRIMARY KEY):** Unique identifier for a treatment plan.
- **Plan_Description (VARCHAR(255)):** Description of the treatment plan.

- **Disease_ID (INT, REFERENCES Disease(Disease_ID))**: ID linking to a specific disease for which the plan is applicable.

Hospital

- **Hospital_ID (SERIAL, PRIMARY KEY)**: Unique identifier for a hospital.
- **Name (VARCHAR(255))**: Name of the hospital.

Healthcare_Provider

- **Provider_ID (SERIAL, PRIMARY KEY)**: Unique identifier for a healthcare provider.
- **Name (VARCHAR(255))**: Name of the healthcare provider.
- **Hospital_ID (INT, REFERENCES Hospital(Hospital_ID))**: ID linking to the hospital where the provider works.

Diagnostic_Test

- **Test_ID (SERIAL, PRIMARY KEY)**: Unique identifier for a diagnostic test.
- **Person_ID (INT, REFERENCES Person(Person_ID))**: ID of the person on whom the test was performed.
- **Disease_ID (INT, REFERENCES Disease(Disease_ID))**: ID of the disease related to the test.

Insurance

- **Insurance_ID (SERIAL, PRIMARY KEY)**: Unique identifier for an insurance record.
- **Provider_Name (VARCHAR(255))**: Name of the insurance provider.
- **Person_ID (INT, REFERENCES Person(Person_ID))**: ID of the person who holds the insurance.

Hospital_Person

- **Hospital_Person_ID (SERIAL, PRIMARY KEY)**: Unique identifier for a hospital-person association.
- **Hospital_ID (INT, REFERENCES Hospital(Hospital_ID))**: ID of the hospital.
- **Person_ID (INT, REFERENCES Person(Person_ID))**: ID of the person associated with the hospital

3.3. Data base schema:

--Creation Queries

CREATE DATABASE DiseaseManagement;

CREATE TABLE Disease_Type (
 Disease_Type_Code CHAR(10) PRIMARY KEY,
 Description VARCHAR(255));

-- Stores different types of diseases to categorize diseases in the Disease table.

```

CREATE TABLE Disease (
    Disease_ID SERIAL PRIMARY KEY,
    Disease_Name VARCHAR(255),
    Intensity_Level INT,
    Source_Disease_Cd INT REFERENCES Disease(Disease_ID),
    Disease_Type_Code CHAR(10) REFERENCES Disease_Type(Disease_Type_Code)
);

```

-- Holds information about specific diseases, including a reference to Disease_Type and possible source diseases.

```

CREATE TABLE Location (
    Location_ID SERIAL PRIMARY KEY,
    City_Name VARCHAR(255),
    State_Name VARCHAR(255),
    Country_Name VARCHAR(255),
    Wealth_Rank_Number INT,
    Developing_Flag BOOLEAN
);

```

-- Details geographic locations, which can be used to analyse disease patterns geographically.

```

CREATE TABLE Race (
    Race_Code CHAR(10) PRIMARY KEY,
    Race_Description VARCHAR(255)
);

```

-- Catalogues races to understand and analyse demographic influences on disease patterns.

```

CREATE TABLE Person (
    Person_ID SERIAL PRIMARY KEY,
    First_Name VARCHAR(255),
    Last_Name VARCHAR(255),
    Primary_Location_ID INT REFERENCES Location(Location_ID),
    Race_CD CHAR(10) REFERENCES Race(Race_Code)
);

```

-- Records individuals, linking them to locations and races for demographic studies.

```

CREATE TABLE Treatment_Plan (
    Treatment_Plan_ID SERIAL PRIMARY KEY,
    Plan_Description VARCHAR(255),
    Disease_ID INT REFERENCES Disease(Disease_ID)
);

```

-- Contains treatment plans linked to specific diseases to manage patient care effectively.

```

CREATE TABLE Hospital (
    Hospital_ID SERIAL PRIMARY KEY,
    Name VARCHAR(255)
);

```

-- Lists hospitals, which are key locations for delivering healthcare and treatments.

```
CREATE TABLE Healthcare_Provider (  
    Provider_ID SERIAL PRIMARY KEY,  
    Name VARCHAR(255),  
    Hospital_ID INT REFERENCES Hospital(Hospital_ID)  
);
```

-- Details healthcare providers, associating them with hospitals to manage their placement and track their employment.

```
CREATE TABLE Diagnostic_Test (  
    Test_ID SERIAL PRIMARY KEY,  
    Person_ID INT REFERENCES Person(Person_ID),  
    Disease_ID INT REFERENCES Disease(Disease_ID)  
);
```

-- Keeps records of diagnostic tests performed on individuals, linking tests to diseases and persons for medical tracking.

```
CREATE TABLE Insurance (  
    Insurance_ID SERIAL PRIMARY KEY,  
    Provider_Name VARCHAR(255),  
    Person_ID INT REFERENCES Person(Person_ID)  
);
```

-- Manages insurance provider data for individuals to facilitate billing and medical service access.

```
CREATE TABLE Hospital_Person (  
    Hospital_Person_ID SERIAL PRIMARY KEY,  
    Hospital_ID INT REFERENCES Hospital(Hospital_ID),  
    Person_ID INT REFERENCES Person(Person_ID)  
);
```

-- Tracks which individuals (patients or staff) are associated with which hospitals.

3.3. Data insertion:

The database has been populated with detailed insert statements for each table, containing realistic data sets to ensure that the system is fully functional and ready to use from the start.

```
INSERT INTO Disease_Type (Disease_Type_Code, Description) VALUES  
('DT001', 'Infectious'),('DT002', 'Chronic'),('DT003', 'Hereditary'),  
('DT004', 'Congenital'),('DT005', 'Autoimmune'),('DT006', 'Neoplastic'),  
('DT007', 'Degenerative'),('DT008', 'Nutritional'),('DT009', 'Psychiatric'),  
('DT010', 'Endocrine'),('DT011', 'Cardiovascular'),('DT012', 'Dermatological'),  
('DT013', 'Gastrointestinal'),('DT014', 'Neurological'),('DT015', 'Respiratory'),  
('DT016', 'Orthopedic'),('DT017', 'Ocular'),('DT018', 'Dental'),  
('DT019', 'Urological'),('DT020', 'Renal');
```

INSERT INTO Disease (Disease_Name, Intensity_Level, Source_Disease_Cd, Disease_Type_Code) VALUES

('Flu', 2, NULL, 'DT001'), ('Diabetes', 6, NULL, 'DT010'),
('Alzheimer"s', 8, NULL, 'DT014'), ('Celiac Disease', 5, NULL, 'DT008'),
('Multiple Sclerosis', 7, NULL, 'DT005'), ('Cancer', 9, NULL, 'DT006'),
('Asthma', 4, NULL, 'DT015'), ('Osteoporosis', 3, NULL, 'DT016'),
('Glaucoma', 4, NULL, 'DT017'), ('Dental Caries', 3, NULL, 'DT018'),
('Kidney Stones', 5, NULL, 'DT020'), ('Heart Disease', 9, NULL, 'DT011'),
('Eczema', 3, NULL, 'DT012'), ('Gastritis', 4, NULL, 'DT013'),
('Epilepsy', 6, NULL, 'DT014'), ('Tuberculosis', 7, NULL, 'DT015'),
('Arthritis', 4, NULL, 'DT016'), ('Conjunctivitis', 2, NULL, 'DT017'),
('Gingivitis', 2, NULL, 'DT018'), ('Bladder Infection', 5, NULL, 'DT019');

INSERT INTO Location (City_Name, State_Name, Country_Name, Wealth_Rank_Number, Developing_Flag) VALUES

('New York', 'New York', 'USA', 9, FALSE), ('Los Angeles', 'California', 'USA', 8, FALSE),
('Chicago', 'Illinois', 'USA', 7, FALSE), ('Houston', 'Texas', 'USA', 7, FALSE),
('Phoenix', 'Arizona', 'USA', 6, FALSE), ('Philadelphia', 'Pennsylvania', 'USA', 6, FALSE),
('San Antonio', 'Texas', 'USA', 6, FALSE), ('San Diego', 'California', 'USA', 7, FALSE),
('Dallas', 'Texas', 'USA', 7, FALSE), ('San Jose', 'California', 'USA', 8, FALSE),
('London', 'London', 'UK', 10, FALSE), ('Manchester', 'Greater Manchester', 'UK', 7, FALSE),
('Birmingham', 'West Midlands', 'UK', 6, FALSE), ('Leeds', 'West Yorkshire', 'UK', 6, FALSE),
('Glasgow', 'Scotland', 'UK', 6, FALSE), ('Paris', 'Ile-de-France', 'France', 10, FALSE),
('Marseille', 'Provence-Alpes-Côte d"Azur', 'France', 7, FALSE),
('Lyon', 'Auvergne-Rhône-Alpes', 'France', 7, FALSE),
('Toulouse', 'Occitanie', 'France', 6, FALSE),
('Nice', 'Provence-Alpes-Côte d"Azur', 'France', 8, FALSE);

INSERT INTO Race (Race_Code, Race_Description) VALUES

('RC001', 'Caucasian'), ('RC002', 'African American'),
('RC003', 'Hispanic'), ('RC004', 'Asian'),
('RC005', 'Native American'), ('RC006', 'Pacific Islander'),
('RC007', 'Middle Eastern'), ('RC008', 'Mixed Race'),
('RC009', 'Other'), ('RC010', 'Indian'),
('RC011', 'Chinese'), ('RC012', 'Japanese'),
('RC013', 'Korean'), ('RC014', 'Vietnamese'),
('RC015', 'Russian'), ('RC016', 'Italian'),
('RC017', 'Greek'), ('RC018', 'French'),
('RC019', 'German'), ('RC020', 'Brazilian');

INSERT INTO Person (First_Name, Last_Name, Primary_Location_ID, Race_CD) VALUES

('John', 'Doe', 1, 'RC001'), ('Jane', 'Smith', 2, 'RC002'),
('Alice', 'Johnson', 3, 'RC003'), ('Bob', 'Williams', 4, 'RC004'),
('Michael', 'Brown', 5, 'RC005'), ('Maria', 'Davis', 6, 'RC006'),

('James', 'Miller', 7, 'RC007'),('Linda', 'Wilson', 8, 'RC008'),
('Robert', 'Moore', 9, 'RC009'),('Patricia', 'Taylor', 10, 'RC010'),
('David', 'Anderson', 11, 'RC011'),('Elizabeth', 'Thomas', 12, 'RC012'),
('Joseph', 'Jackson', 13, 'RC013'),('Jennifer', 'White', 14, 'RC014'),
('Charles', 'Harris', 15, 'RC015'),('Barbara', 'Martin', 16, 'RC016'),
('Thomas', 'Thompson', 17, 'RC017'),('Susan', 'Garcia', 18, 'RC018'),
('Christopher', 'Martinez', 19, 'RC019'),('Jessica', 'Robinson', 20, 'RC020');

INSERT INTO Treatment_Plan (Plan_Description, Disease_ID) VALUES

('Standard Flu Treatment', 1),('Type 2 Diabetes Management', 2),
('Alzheimer"s Care Plan', 3),('Gluten-Free Diet', 4),
('MS Immunomodulators Therapy', 5),('Cancer Chemotherapy', 6),
('Asthma Inhalers Plan', 7),('Calcium and Vitamin D Supplements', 8),
('Eye Drops for Glaucoma', 9),('Tooth Filling and Fluoride Treatment', 10),
('Fluid Intake and Dietary Changes', 11),('Lifestyle and Medication for Heart', 12),
('Moisturizers and Topical Steroids', 13),('Antacids and Diet Management', 14),
('Anti-Seizure Medication Plan', 15),('TB Antibiotics Course', 16),
('Joint Pain Relief Measures', 17),('Eye Irritation Soothing Drops', 18),
('Gum Disease Antibacterial Rinse', 19),('Antibiotics for Bladder Infection', 20);

INSERT INTO Hospital (Name) VALUES

('Saint Mary"s Hospital'),('Mount Sinai Hospital'),
('Cleveland Clinic'),('Johns Hopkins Hospital'),
('Mayo Clinic'),('Massachusetts General Hospital'),
('UCLA Medical Center'),('New York-Presbyterian Hospital'),
('University of Michigan Hospitals'),('UCSF Medical Center'),
('Imperial College Healthcare NHS'),('King"s College Hospital'),
('Guy"s and St Thomas" NHS Foundation'),('Royal Free Hospital'),
('The Royal London Hospital'),('Hôpital Européen Georges-Pompidou'),
('Pitié-Salpêtrière Hospital'),('Hôpital Saint-Louis'),
('Hôpital Cochin'),('Centre Hospitalier Universitaire de Nice');

INSERT INTO Healthcare_Provider (Name, Hospital_ID) VALUES

('Dr. Sarah Connors', 1),('Dr. John Hart', 1),
('Dr. Lisa Ray', 2),('Dr. Omar Little', 2),
('Dr. Jessica Alba', 3),('Dr. Bruce Banner', 3),
('Dr. Clark Kent', 4),('Dr. Diana Prince', 4),
('Dr. Jean Grey', 5),('Dr. Charles Xavier', 5),
('Dr. Stephen Strange', 6),('Dr. Hank Pym', 6),
('Dr. Tony Stark', 7),('Dr. Natasha Romanoff', 7),
('Dr. Steve Rogers', 8),('Dr. Peter Parker', 8),
('Dr. Bruce Wayne', 9),('Dr. Clark Kent', 9),
('Dr. Lois Lane', 10),('Dr. Barry Allen', 10);

INSERT INTO Diagnostic_Test (Person_ID, Disease_ID) VALUES

(1, 1),(2, 2),(3, 3),(4, 4),(5, 5),(6, 6),(7, 7),(8, 8),

(9, 9),(10, 10),(11, 11),(12, 12),(13, 13),(14, 14),
(15, 15),(16, 16),(17, 17),(18, 18),(19, 19),(20, 20);

INSERT INTO Insurance (Provider_Name, Person_ID) VALUES

('UnitedHealth', 1),('Kaiser Foundation', 2),
('WellPoint Inc. Group', 3),('Aetna Group', 4),
('Humana Group', 5),('HCSC Group', 6),
('Cigna Health Group', 7),('Highmark Group', 8),
('Coventry Corp. Group', 9),('HIP Insurance Group', 10),
('Independence Blue Cross Group', 11),
('Blue Shield of CA Group', 12),
('Blue KC Group', 13),('CareFirst Inc. Group', 14),
('Health Net of California', 15),('Centene Corp. Group', 16),
('California Physicians' Service, 17),('Metropolitan Group', 18),
('Molina Healthcare Inc. Group', 19),('Wellcare Group', 20);

3.4. DML Queries:

1) Before adding a new diagnostic test, we need to ensure that the Person_ID exists or we need to add a new person with Person_ID 21 to the Person table first.

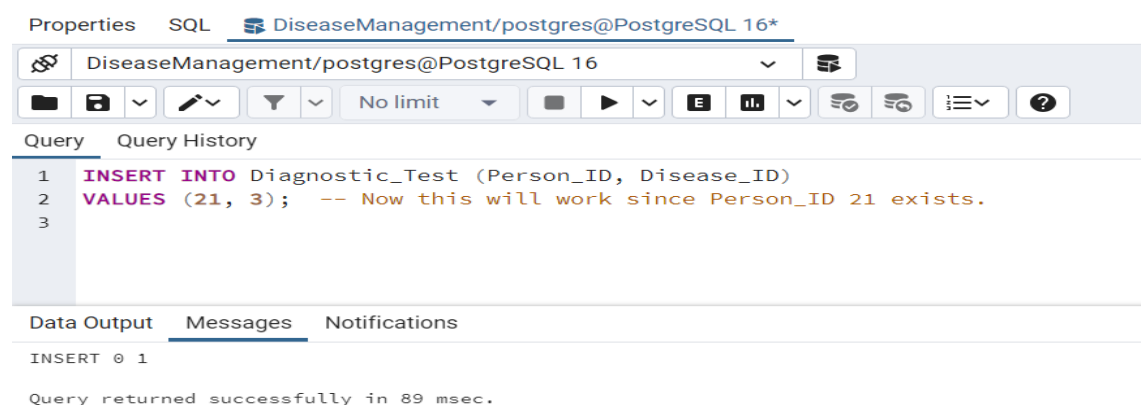
INSERT INTO Person (First_Name, Last_Name, Primary_Location_ID, Race_CD)
VALUES ('New', 'Person', 1, 'RC001'); -- Location_ID 1 and Race_CD 'RC001' exists.

--It returned no results; it confirms that there is no such person in our database.

Now, inserting a New Person before we can add a diagnostic test for them.

INSERT INTO Person (First_Name, Last_Name, Primary_Location_ID, Race_CD)
VALUES ('New', 'Person', 1, 'RC001'); -- Assuming Location_ID 1 and Race_CD 'RC001'
exist.

INSERT INTO Diagnostic_Test (Person_ID, Disease_ID)
VALUES (21, 3); -- Now this will work since Person_ID 21 exists.



Purpose: Adds a record for a new diagnostic test performed on a patient with Person_ID 21 for the disease with Disease_ID 3.

Utility: It is critical for updating the database with new diagnostic events as they occur, keeping the patient's health records current.).

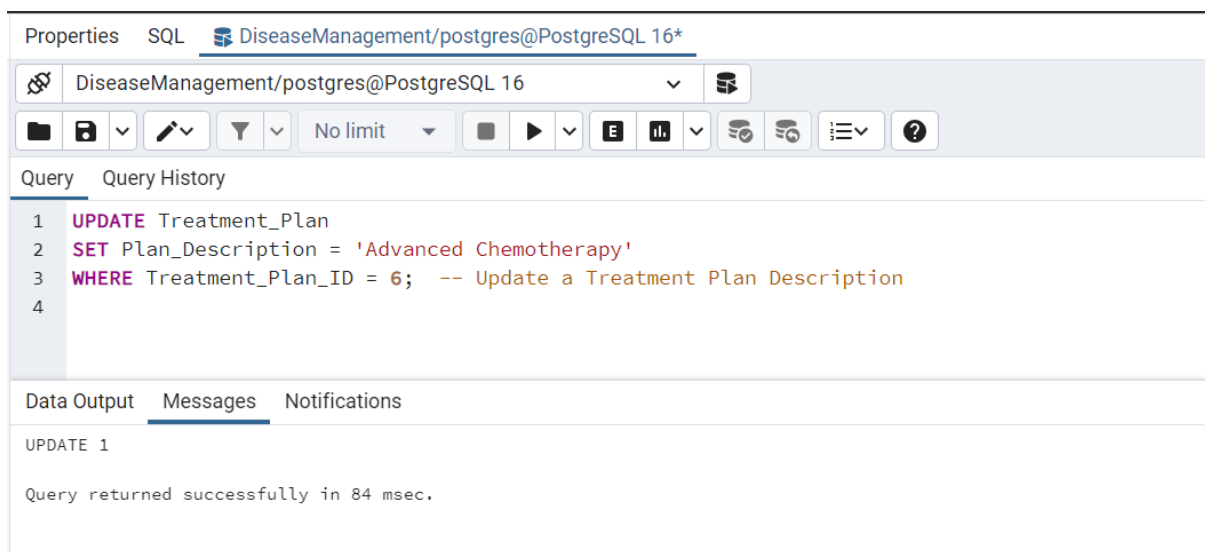
2) Updating a Treatment plan:

--2)

UPDATE Treatment_Plan

SET Plan_Description = 'Advanced Chemotherapy'

WHERE Treatment_Plan_ID = 6; -- Update a Treatment Plan Description



Purpose: Updates the description of an existing treatment plan to reflect changes in treatment strategy or specifics.

Utility: Ensures that treatment plans are up-to-date and accurately reflect current medical practice, which is vital for effective patient care.

3) Deleting an outdated Insurance record:

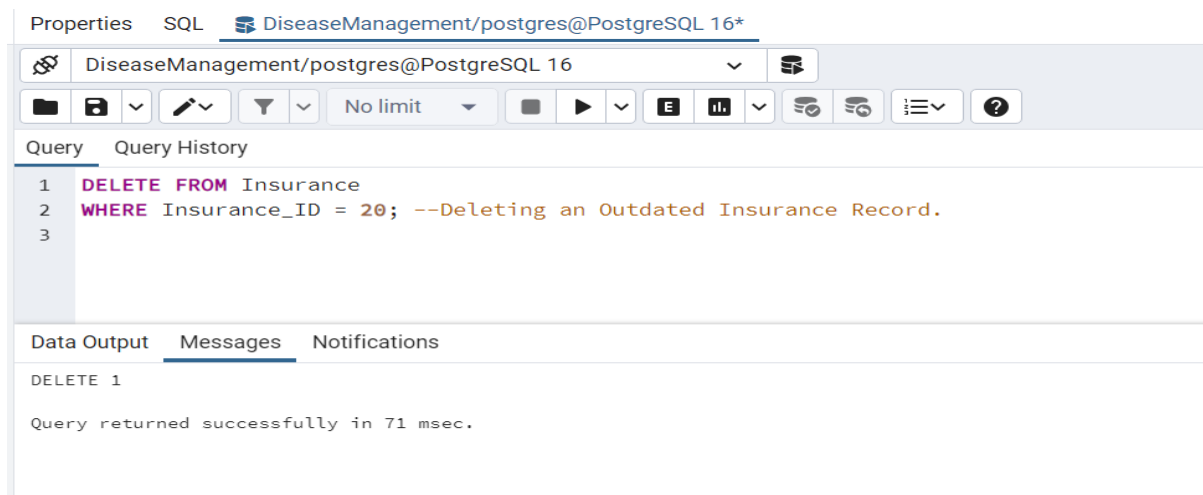
When insurance details become outdated or irrelevant, they need to be removed from the database to prevent confusion and ensure that the most current information is used.

--3)

DELETE FROM Insurance

WHERE Insurance_ID = 20; --Deleting an Outdated Insurance Record.

Utility: Keeps the database clean of old data, which can help prevent confusion and ensure that current insurance information is used for billing and administrative processes.



Purpose: Removes an outdated or no longer relevant insurance record from the database.

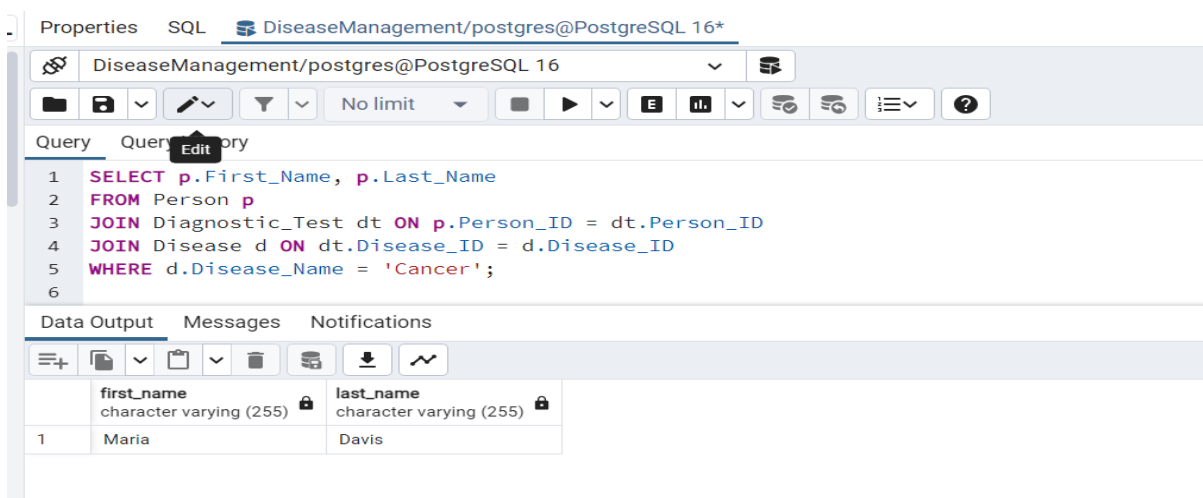
4) RUNNING OPERATIONAL QUERIES:

Operational queries play a crucial role in retrieving information that supports the daily operations of a database.

Here are some examples of operational queries, along with their purpose:

1)Query to Retrieve Patients with a Specific Disease:

```
SELECT p.First_Name, p.Last_Name
FROM Person p
JOIN Diagnostic_Test dt ON p.Person_ID = dt.Person_ID
JOIN Disease d ON dt.Disease_ID = d.Disease_ID
WHERE d.Disease_Name = 'Cancer';
```

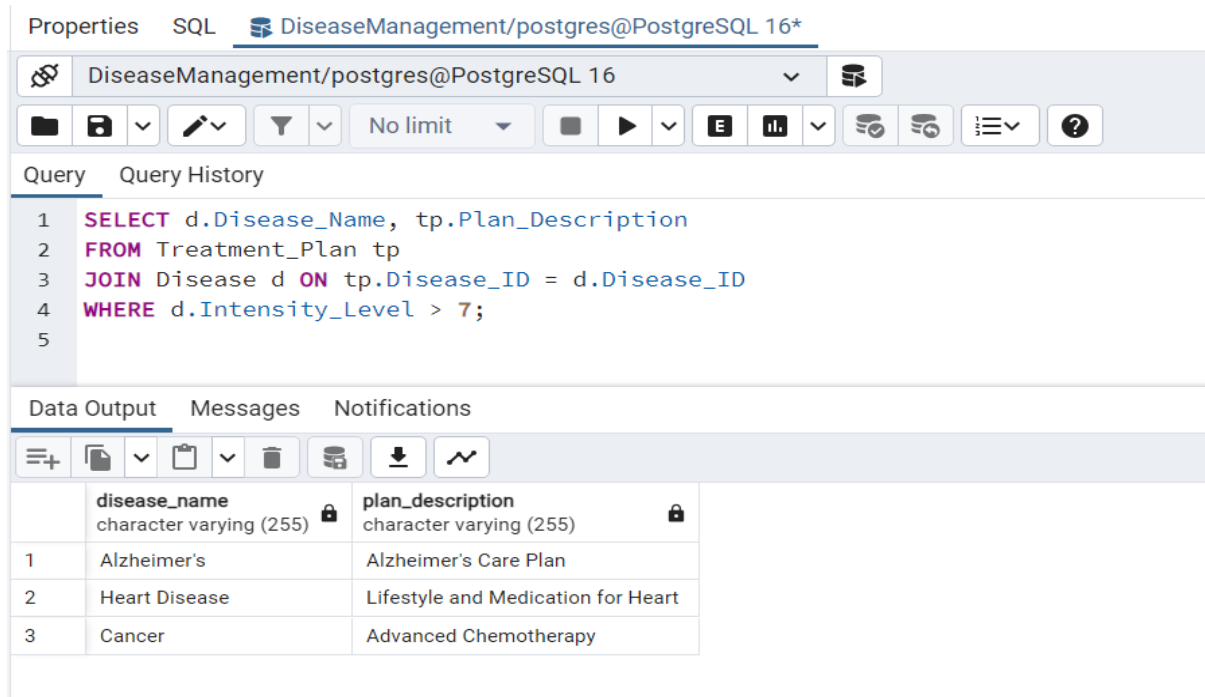


Purpose: This query retrieves a list of patients diagnosed with Cancer.

Utility: It is useful for healthcare providers to identify patients for targeted treatments, resource planning, and patient outreach programs. It is also beneficial for reporting purposes and to monitor the prevalence of Cancer within the patient population.

2)Query to List All Treatment Plans for Diseases with High Intensity:

```
SELECT d.Disease_Name, tp.Plan_Description
FROM Treatment_Plan tp
JOIN Disease d ON tp.Disease_ID = d.Disease_ID
WHERE d.Intensity_Level > 7;
```



The screenshot shows a PostgreSQL query editor interface. The top bar indicates the connection is 'DiseaseManagement/postgres@PostgreSQL 16*'. The query editor contains the following SQL query:

```
1 SELECT d.Disease_Name, tp.Plan_Description
2 FROM Treatment_Plan tp
3 JOIN Disease d ON tp.Disease_ID = d.Disease_ID
4 WHERE d.Intensity_Level > 7;
5
```

Below the query editor, the 'Data Output' tab is active, displaying the results of the query in a table format:

	disease_name character varying (255)	plan_description character varying (255)
1	Alzheimer's	Alzheimer's Care Plan
2	Heart Disease	Lifestyle and Medication for Heart
3	Cancer	Advanced Chemotherapy

Purpose: Retrieves all treatment plans for diseases classified as high intensity (severity level greater than 7).

Utility: Useful for medical staff to prioritize care and allocate resources efficiently. It helps in understanding the severity of cases and in tailoring medical responses accordingly.

3) Find Hospitals in a Specific Location:

```
SELECT h.Name, l.City_Name
FROM Hospital h
JOIN Location l ON l.Location_ID = h.Location_ID
WHERE l.State_Name = 'California';
```

Purpose: Identifies all hospitals in the state of California.

Utility: Aids in geographical analysis of healthcare facilities, useful for logistics, patient referrals, or even for patients looking for hospitals in their vicinity.

Properties SQL DiseaseManagement/postgres@PostgreSQL 16*

DiseaseManagement/postgres@PostgreSQL 16

Query Query History

```

1 SELECT h.Name, l.City_Name
2 FROM Hospital h
3 JOIN Location l ON l.Location_ID = l.Location_ID
4 WHERE l.State_Name = 'California'

```

Data Output Messages Notifications

	name character varying (255)	city_name character varying (255)
1	Saint Mary's Hospital	Los Angeles
2	Mount Sinai Hospital	Los Angeles
3	Cleveland Clinic	Los Angeles
4	Johns Hopkins Hospital	Los Angeles
5	Mayo Clinic	Los Angeles
6	Massachusetts General Hospital	Los Angeles
7	UCLA Medical Center	Los Angeles
8	New York-Presbyterian Hospital	Los Angeles
9	University of Michigan Hospitals	Los Angeles
10	UCSF Medical Center	Los Angeles
11	Imperial College Healthcare NHS	Los Angeles
12	King's College Hospital	Los Angeles
13	Guy's and St Thomas' NHS Foundation	Los Angeles
14	Royal Free Hospital	Los Angeles
15	The Royal London Hospital	Los Angeles
16	Hôpital Européen Georges-Pompidou	Los Angeles
17	Pitié-Salpêtrière Hospital	Los Angeles

✓ Successfully run. Total query runtime: 98 msec. 60 rows affected. ✕

Total rows: 60 of 60 Query complete 00:00:00.098 Ln 6, Col

5. Generating Reports for Analysis:

Reports is an essential step in analysing the data that has been collected and stored in a database. These reports provide valuable insights and information that can help individuals and organizations make informed decision.

5.1) Report on disease frequency by location:

--Query:

```

SELECT l.City_Name, d.Disease_Name, COUNT(*) as Frequency
FROM Diagnostic_Test dt
JOIN Disease d ON dt.Disease_ID = d.Disease_ID
JOIN Person p ON dt.Person_ID = p.Person_ID
JOIN Location l ON p.Primary_Location_ID = l.Location_ID
GROUP BY l.City_Name, d.Disease_Name
ORDER BY Frequency DESC;

```

Purpose: Provides a report on the frequency of each disease by city.

Utility: Useful for public health analysis and resource allocation. Helps in identifying disease hotspots and could inform public health initiatives and prevention strategies.

Properties SQL DiseaseManagement/postgres@PostgreSQL 16*

DiseaseManagement/postgres@PostgreSQL 16

Query Query History

```

2 FROM Diagnostic_Test dt
3 JOIN Disease d ON dt.Disease_ID = d.Disease_ID
4 JOIN Person p ON dt.Person_ID = p.Person_ID
5 JOIN Location l ON p.Primary_Location_ID = l.Location_ID
6 GROUP BY l.City_Name, d.Disease_Name
7 ORDER BY Frequency DESC;

```

Data Output Messages Notifications

	city_name character varying (255)	disease_name character varying (255)	frequency bigint
1	New York	Alzheimer's	3
2	Phoenix	Multiple Sclerosis	1
3	Marseille	Arthritis	1
4	Paris	Tuberculosis	1
5	Chicago	Alzheimer's	1
6	Nice	Bladder Infection	1
7	Glasgow	Epilepsy	1
8	San Antonio	Asthma	1
9	New York	Flu	1
10	Birmingham	Eczema	1
11	Manchester	Heart Disease	1
12	Dallas	Glaucoma	1
13	Philadelphia	Cancer	1
14	London	Kidney Stones	1
15	Houston	Celiac Disease	1
16	Los Angeles	Diabetes	1

✓ Successfully run. Total query runtime: 93 msec. 21 rows affected. ✕

Total rows: 21 of 21 Query complete 00:00:00.093 Ln 8, Col 1

5.2. Report on healthcare provider workload:

--Query:

```

SELECT hp.Name, COUNT(*) as NumberOfPatients
FROM Healthcare_Provider hp
JOIN Hospital_Person hp2 ON hp.Hospital_ID = hp2.Hospital_ID
GROUP BY hp.Name
ORDER BY NumberOfPatients DESC;

```

Properties SQL DiseaseManagement/postgres@PostgreSQL 16*

DiseaseManagement/postgres@PostgreSQL 16

Query Query History

```

1 SELECT hp.Name, COUNT(*) as NumberOfPatients
2 FROM Healthcare_Provider hp
3 JOIN Hospital_Person hp2 ON hp.Hospital_ID = hp2.Hospital_ID
4 GROUP BY hp.Name
5 ORDER BY NumberOfPatients DESC;

```

Data Output Messages Notifications

	name character varying (255)	numberofpatients bigint
1	Dr. Clark Kent	4
2	Dr. Natasha Romanoff	2
3	Dr. Jean Grey	2
4	Dr. Steve Rogers	2
5	Dr. Bruce Banner	2
6	Dr. Peter Parker	2
7	Dr. Omar Little	2
8	Dr. Bruce Wayne	2
9	Dr. Charles Xavier	2
10	Dr. Hank Pym	2
11	Dr. Lisa Ray	2
12	Dr. Lois Lane	2
13	Dr. Stephen Strange	2
14	Dr. John Hart	2
15	Dr. Diana Prince	2
16	Dr. Tony Stark	2

✓ Successfully run. Total query runtime: 124 msec. 19 rows affected. ✕

Total rows: 19 of 19 Query complete 00:00:00.124 Ln 5, Col 32

Purpose: Summarizes the number of patients each healthcare provider is handling, which can help in assessing workload and resource allocation.

Utility: Assists in managing healthcare provider workloads and can be instrumental in staffing decisions. It is also beneficial for analysing the distribution of patient care among providers.

Each of these queries plays an essential role in the operational, transactional, and analytical functions of the database. They collectively serve to maintain the database's integrity, provide valuable insights, and ensure that the database is a useful tool for the institution it serves.

6. Dimensional modelling for disease management system:

To create a dimensional model that addresses the business problem of enhancing disease management, tracking, diagnosis, and treatment by following Ralph Kimball methodology, which is centred on designing a star schema. This involves identifying the fact tables that capture business event and dimension tables that provide context to the facts.

i) Identifying the business processes:

In the context of the business problem, the main business processes that will be represented as fact tables in the dimensional model are:

Disease Diagnosis: Captures each instance of disease diagnosis.

Treatment Administration: Records details of treatments provided to patients, tracking the administration and outcomes of these treatments.

(ii) Defining Fact tables:

Based on the identified business processes, defining the following fact tables:

1) FactDiagnosis:

Primary Measures: Counts of diagnoses, potentially extended by severity or urgency indicators.

Foreign Keys: Linking to dimensions such as Date, Patient, Disease, and DiagnosticTest.

Purpose: Tracks each diagnosis event, providing insights into when, where and how diseases are diagnosed, linked to time, patient, disease type, and diagnostic methods used.

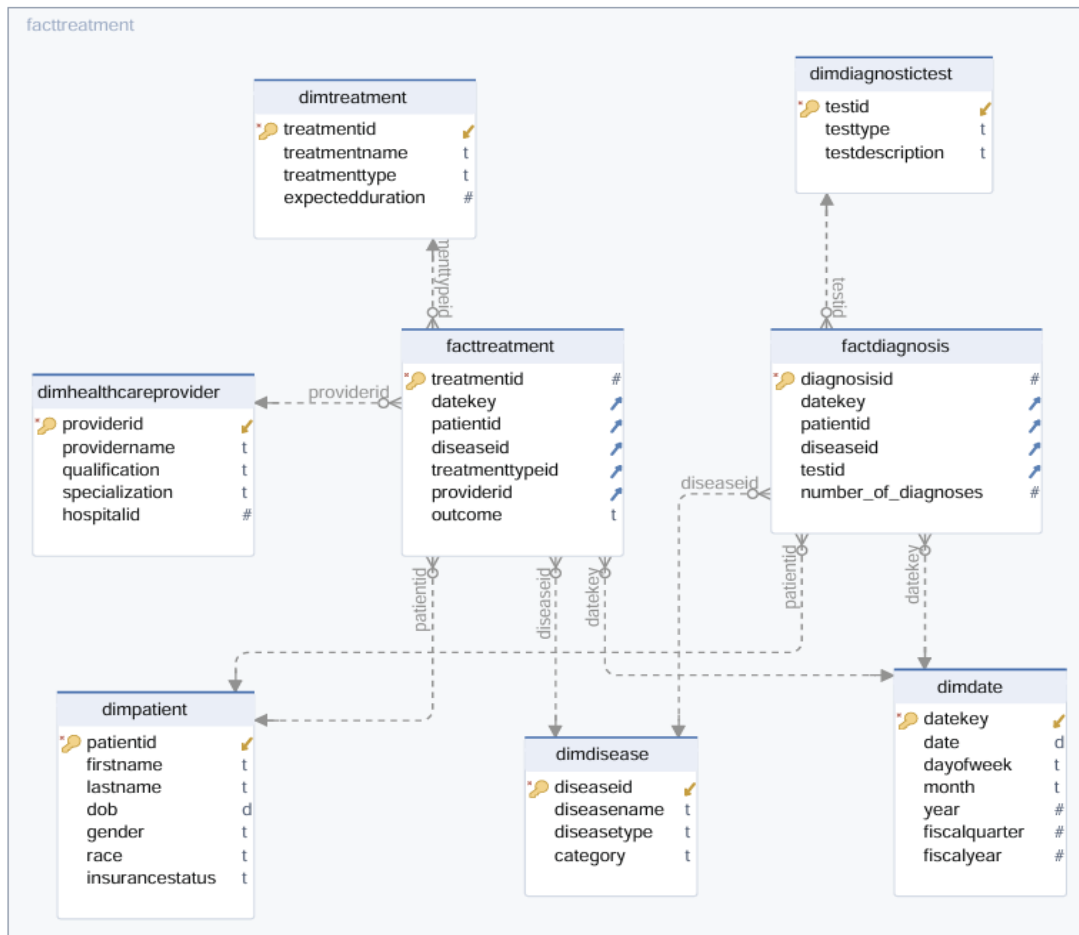
2) FactTreatment:

Primary Measures: Number of treatments, duration of treatment courses, treatment outcomes.

Foreign Keys: Link to dimensions such as Date, Patient, Disease, Treatment, and HealthcareProvider.

Purpose: Captures detailed records on treatments administered including the outcomes, and the providers involved, crucial for effectiveness and quality of care analysis.

DIMENSIONAL MODEL



(iii) Defining Dimension Tables:

The following dimension tables will provide the descriptive attributes related to each fact table:

1) Date Dimension (DimDate):

Attributes: DateKey, Date, DayOfWeek, Month, Year, FiscalQuarter, FiscalYear.

Purpose: This table provides temporal context for all events in the data warehouse, facilitating time-series analysis. This dimension helps analyze data across different timeframes.

2) Patient Dimension (DimPatient):

Attributes: PatientID, FirstName, LastName, DOB, Gender, Race, InsuranceStatus.

Purpose: Tracks demographic information about patients, supporting detailed patient-level analysis. Provides a comprehensive view of the patient demographics.

3) Disease Dimension (DimDisease):

Attributes: DiseaseID, DiseaseName, DiseaseType, Category.

Purpose: Facilitates analysis by disease type and category, crucial for epidemiological studies. Categorizes diseases which can be useful for specialized analytics.

4) Diagnostic Test Dimension (DimDiagnosticTest):

Attributes: TestID, TestType, TestDescription.

Purpose: Provides details on diagnostic tests, aiding in the evaluation of test utilization and effectiveness. Describes the tests used in diagnostics, important for analysis of test utilization and efficacy.

5) Treatment Dimension (DimTreatment):

Attributes: TreatmentID, TreatmentName, TreatmentType, ExpectedDuration.

Purpose: Enables analysis of treatment patterns and their outcomes. Detailed records of treatment protocols provide insights into treatment patterns and outcomes.

6) Healthcare Provider Dimension (DimHealthcareProvider):

Attributes: ProviderID, ProviderName, Qualification, Specialization, HospitalID.

Purpose: Stores information about healthcare providers, important for performance and resource allocation analysis. Helps in analysing performance and specialization areas of providers.

7) Setting up data warehouse in PostgreSQL:

It involves several steps, from creating a new schema dedicated to analytical data to defining the tables based on the dimensional model .

Creating a New Schema in PostgreSQL:

Starting by creating a separate schema within our existing DiseaseManagement database. This schema will house all our data warehouse tables.

Purpose: Isolating the data warehouse in its own schema helps maintain clean separation from the operational data, simplifies security and access management, and organizes the analytical environment.

--Creation Queries:

CREATE SCHEMA DataWarehouse;

--DIMENSION TABLES:

```
CREATE TABLE DataWarehouse.DimDate (  
    DateKey SERIAL PRIMARY KEY,  
    Date DATE,  
    DayOfWeek VARCHAR(10),  
    Month VARCHAR(10),  
    Year INT,  
    FiscalQuarter INT,  
    FiscalYear INT );
```


CREATE TABLE DataWarehouse.DimPatient (

PatientID INT PRIMARY KEY,
FirstName VARCHAR(255),
LastName VARCHAR(255),
DOB DATE,
Gender VARCHAR(10),
Race VARCHAR(50),
InsuranceStatus VARCHAR(50));

CREATE TABLE DataWarehouse.DimDisease (

DiseaseID INT PRIMARY KEY,
DiseaseName VARCHAR(255),
DiseaseType VARCHAR(100),
Category VARCHAR(100));

CREATE TABLE DataWarehouse.DimDiagnosticTest (

TestID INT PRIMARY KEY,
TestType VARCHAR(100),
TestDescription VARCHAR(255));

CREATE TABLE DataWarehouse.DimTreatment (

TreatmentID INT PRIMARY KEY,
TreatmentName VARCHAR(255),
TreatmentType VARCHAR(100),
ExpectedDuration INT);

CREATE TABLE DataWarehouse.DimHealthcareProvider (

ProviderID INT PRIMARY KEY,
ProviderName VARCHAR(255),
Qualification VARCHAR(255),
Specialization VARCHAR(255),
HospitalID INT);

--FACT TABLES

CREATE TABLE DataWarehouse.FactDiagnosis (

DiagnosisID SERIAL PRIMARY KEY,
DateKey INT REFERENCES DataWarehouse.DimDate(DateKey),
PatientID INT REFERENCES DataWarehouse.DimPatient(PatientID),
DiseaseID INT REFERENCES DataWarehouse.DimDisease(DiseaseID),
TestID INT REFERENCES DataWarehouse.DimDiagnosticTest(TestID),
Number_of_Diagnoses INT);

CREATE TABLE DataWarehouse.FactTreatment (

TreatmentID SERIAL PRIMARY KEY,
DateKey INT REFERENCES DataWarehouse.DimDate(DateKey),

```
PatientID INT REFERENCES DataWarehouse.DimPatient(PatientID),
DiseaseID INT REFERENCES DataWarehouse.DimDisease(DiseaseID),
TreatmentTypeID INT REFERENCES DataWarehouse.DimTreatment(TreatmentID),
ProviderID INT REFERENCES DataWarehouse.DimHealthcareProvider(ProviderID),
Outcome VARCHAR(255) );
```

7.1) ETL Process:

To create ETL (Extract, Transform, Load) processes for transferring data from operational database (OLTP) to our data warehouse in PostgreSQL, we will be utilizing server-side SQL. This step involves extracting data from the transactional system, transforming it into a format suitable for analytical processing, and loading it into our data warehouse schema. Let us dive in detail to set up these ETL processes for each dimension and fact table as defined in our dimensional model.

--Insertion Queries for Dimension Model: Dimension and Fact Tables

--DIMENSION TABLES:

INSERT INTO DataWarehouse.DimDate (DateKey, Date, DayOfWeek, Month, Year, FiscalQuarter, FiscalYear)

SELECT DISTINCT

```
TO_CHAR(g.date, 'YYYYMMDD')::INT AS DateKey,
g.date AS Date,
TO_CHAR(g.date, 'Day') AS DayOfWeek,
TO_CHAR(g.date, 'Month') AS Month,
EXTRACT(YEAR FROM g.date) AS Year,
EXTRACT(QUARTER FROM g.date) AS FiscalQuarter,
EXTRACT(YEAR FROM g.date) AS FiscalYear
```

FROM generate_series('2021-01-01'::date, '2024-12-31'::date, '1 day') AS g(date)

ON CONFLICT (DateKey) DO NOTHING

-- This prevents duplication if some dates are already present

LIMIT 20; -- This limits the number of entries to 20.

INSERT INTO DataWarehouse.DimPatient (PatientID, FirstName, LastName, DOB, Gender, Race, InsuranceStatus)

```
SELECT Person_ID, First_Name, Last_Name, NULL AS DOB, NULL AS Gender,
Race_CD, NULL AS InsuranceStatus
FROM public.person;
```

INSERT INTO DataWarehouse.DimDisease (DiseaseID, DiseaseName, DiseaseType, Category)

```
SELECT Disease_ID, Disease_Name, Disease_Type_Code, NULL AS Category
FROM public.Disease;
```

INSERT INTO DataWarehouse.DimDiagnosticTest (TestID, TestType, TestDescription)

```
SELECT Test_ID, NULL AS TestType, NULL AS TestDescription
FROM public.Diagnostic_Test;
```

```
INSERT INTO DataWarehouse.DimTreatment (TreatmentID, TreatmentName,
TreatmentType, ExpectedDuration)
SELECT Treatment_Plan_ID, Plan_Description, NULL AS TreatmentType, NULL AS
ExpectedDuration
FROM public.Treatment_Plan;
```

```
INSERT INTO DataWarehouse.DimHealthcareProvider (ProviderID, ProviderName,
Qualification, Specialization, HospitalID)
SELECT Provider_ID, Name, NULL AS Qualification, NULL AS Specialization,
Hospital_ID
FROM public.Healthcare_Provider;
```

--FACT TABLES

```
INSERT INTO DataWarehouse.FactDiagnosis (DiagnosisID, DateKey, PatientID,
DiseaseID, TestID, Number_of_Diagnoses)
SELECT Test_ID, 20240101, Person_ID, Disease_ID, Test_ID, COUNT(*)
FROM public.Diagnostic_Test
GROUP BY Test_ID, Person_ID, Disease_ID;
```

```
INSERT INTO DataWarehouse.FactTreatment (TreatmentID, DateKey, PatientID,
DiseaseID, TreatmentTypeID, ProviderID, Outcome)
SELECT Treatment_Plan_ID, 20240101, NULL AS PatientID, Disease_ID,
Treatment_Plan_ID, NULL AS ProviderID, NULL AS Outcome
FROM public.Treatment_Plan;
```

8) Analytical Queries:

Let's proceed to run a series of analytical queries that showcase our data. These queries will highlight the functionality of our data warehouse and demonstrate how it can support decision-making processes in disease management, diagnosis, and treatment.

1) Disease Incidence by Age Group and Gender :

--Query:

```
SELECT
    dd.DiseaseName,
    CASE
        WHEN age(dp.dob) BETWEEN '0 years' AND '18 years' THEN 'Children'
        WHEN age(dp.dob) BETWEEN '19 years' AND '60 years' THEN 'Adults'
        ELSE 'Seniors'
```

```

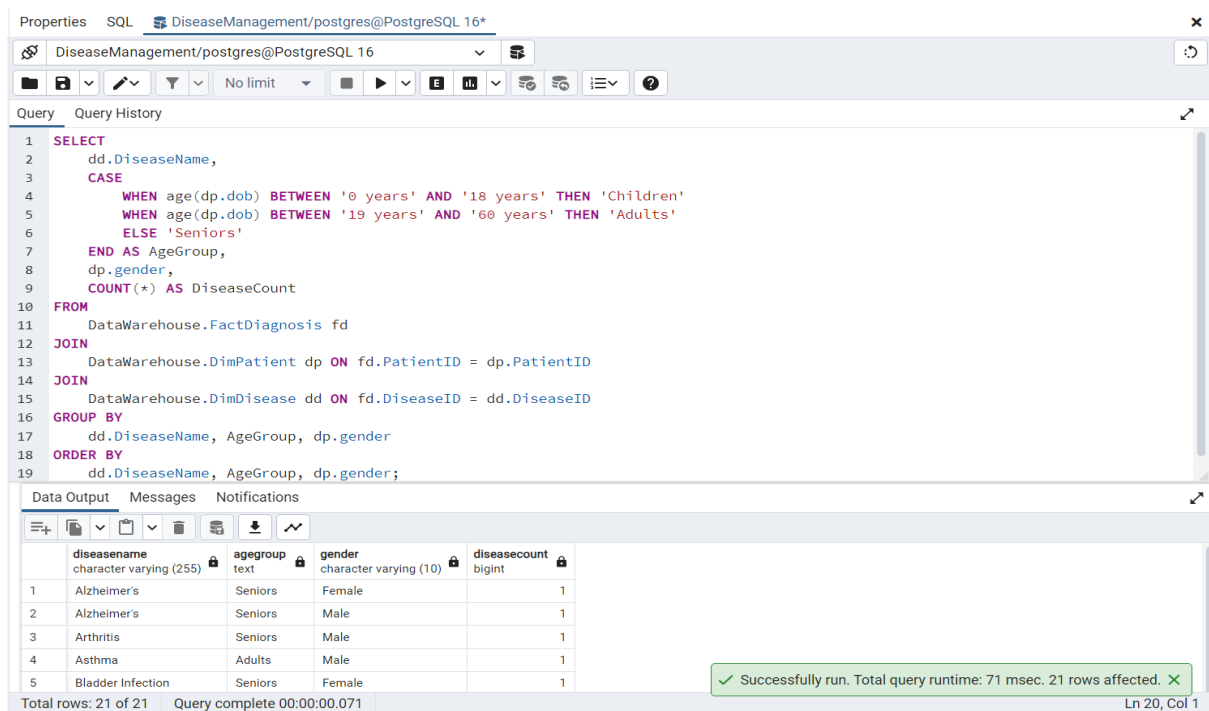
    END AS AgeGroup,
    dp.gender,
    COUNT(*) AS DiseaseCount
FROM
    DataWarehouse.FactDiagnosis fd
JOIN
    DataWarehouse.DimPatient dp ON fd.PatientID = dp.PatientID
JOIN
    DataWarehouse.DimDisease dd ON fd.DiseaseID = dd.DiseaseID
GROUP BY
    dd.DiseaseName, AgeGroup, dp.gender
ORDER BY
    dd.DiseaseName, AgeGroup, dp.gender;

```

Purpose:

This query will provide insights into which age groups and genders are most affected by specific diseases, helping to tailor preventive health programs.

Below is the screen capture of the executed query which retrieved 21 entries of data successfully from the DiseaseManagement database.



The screenshot shows a database query editor with the following SQL query:

```

1 SELECT
2     dd.DiseaseName,
3     CASE
4         WHEN age(dp.dob) BETWEEN '0 years' AND '18 years' THEN 'Children'
5         WHEN age(dp.dob) BETWEEN '19 years' AND '60 years' THEN 'Adults'
6         ELSE 'Seniors'
7     END AS AgeGroup,
8     dp.gender,
9     COUNT(*) AS DiseaseCount
10 FROM
11     DataWarehouse.FactDiagnosis fd
12 JOIN
13     DataWarehouse.DimPatient dp ON fd.PatientID = dp.PatientID
14 JOIN
15     DataWarehouse.DimDisease dd ON fd.DiseaseID = dd.DiseaseID
16 GROUP BY
17     dd.DiseaseName, AgeGroup, dp.gender
18 ORDER BY
19     dd.DiseaseName, AgeGroup, dp.gender;

```

The results are displayed in a table with the following columns: diseasename, agegroup, gender, and diseasecount. The first 5 rows are visible:

	diseasename	agegroup	gender	diseasecount
1	Alzheimer's	Seniors	Female	1
2	Alzheimer's	Seniors	Male	1
3	Arthritis	Seniors	Male	1
4	Asthma	Adults	Male	1
5	Bladder Infection	Seniors	Female	1

The status bar at the bottom indicates: Total rows: 21 of 21, Query complete 00:00:00.071, and a success message: Successfully run. Total query runtime: 71 msec. 21 rows affected.

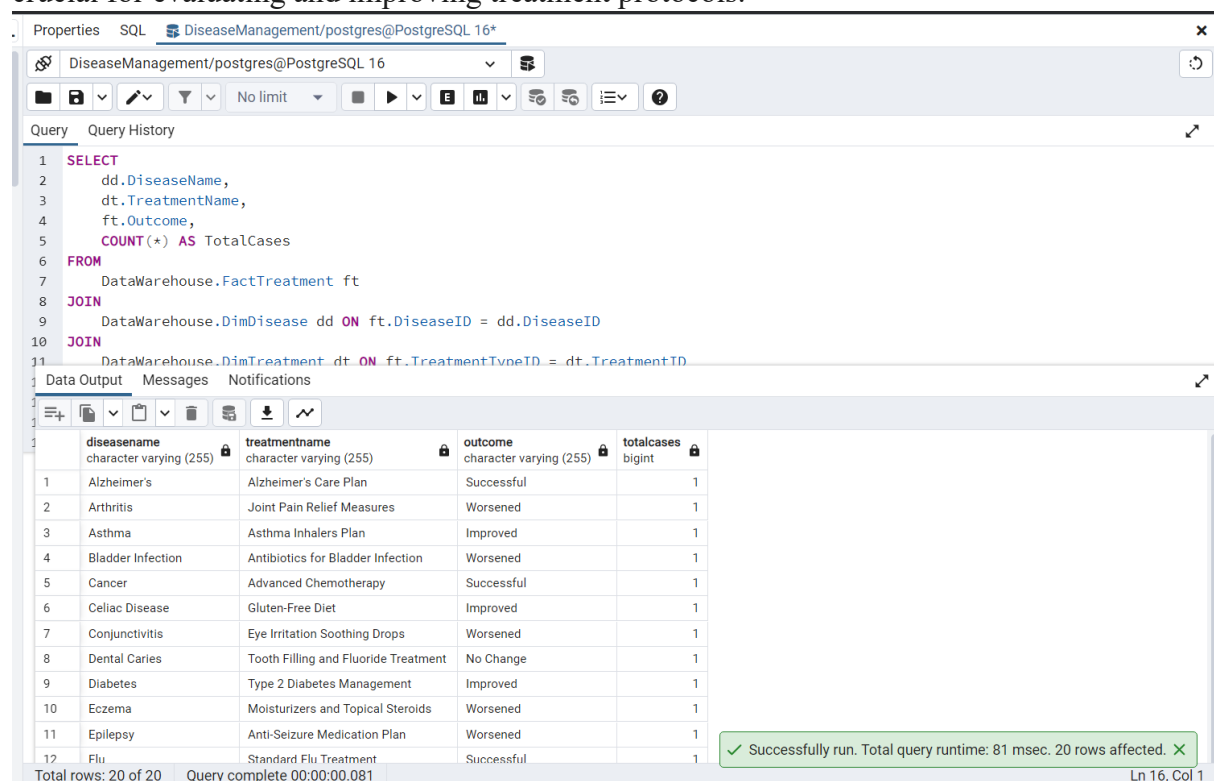
2) Treatment Outcomes by Disease and Treatment Type :

--Query:

```
SELECT
    dd.DiseaseName,
    dt.TreatmentName,
    ft.Outcome,
    COUNT(*) AS TotalCases
FROM
    DataWarehouse.FactTreatment ft
JOIN
    DataWarehouse.DimDisease dd ON ft.DiseaseID = dd.DiseaseID
JOIN
    DataWarehouse.DimTreatment dt ON ft.TreatmentTypeID = dt.TreatmentID
GROUP BY
    dd.DiseaseName, dt.TreatmentName, ft.Outcome
ORDER BY
    dd.DiseaseName, dt.TreatmentName, ft.Outcome;
```

Purpose:

This query explores the effectiveness of different treatments for various diseases, which can be crucial for evaluating and improving treatment protocols.



The screenshot shows a PostgreSQL query editor with the following query:

```
1 SELECT
2     dd.DiseaseName,
3     dt.TreatmentName,
4     ft.Outcome,
5     COUNT(*) AS TotalCases
6 FROM
7     DataWarehouse.FactTreatment ft
8 JOIN
9     DataWarehouse.DimDisease dd ON ft.DiseaseID = dd.DiseaseID
10 JOIN
11     DataWarehouse.DimTreatment dt ON ft.TreatmentTypeID = dt.TreatmentID
```

The results are displayed in a table with the following columns: diseaseName, treatmentname, outcome, and totalcases. The table contains 12 rows of data.

diseaseName	treatmentname	outcome	totalcases
Alzheimer's	Alzheimer's Care Plan	Successful	1
Arthritis	Joint Pain Relief Measures	Worsened	1
Asthma	Asthma Inhalers Plan	Improved	1
Bladder Infection	Antibiotics for Bladder Infection	Worsened	1
Cancer	Advanced Chemotherapy	Successful	1
Celiac Disease	Gluten-Free Diet	Improved	1
Conjunctivitis	Eye Irritation Soothing Drops	Worsened	1
Dental Caries	Tooth Filling and Fluoride Treatment	No Change	1
Diabetes	Type 2 Diabetes Management	Improved	1
Eczema	Moisturizers and Topical Steroids	Worsened	1
Epilepsy	Anti-Seizure Medication Plan	Worsened	1
Flu	Standard Flu Treatment	Successful	1

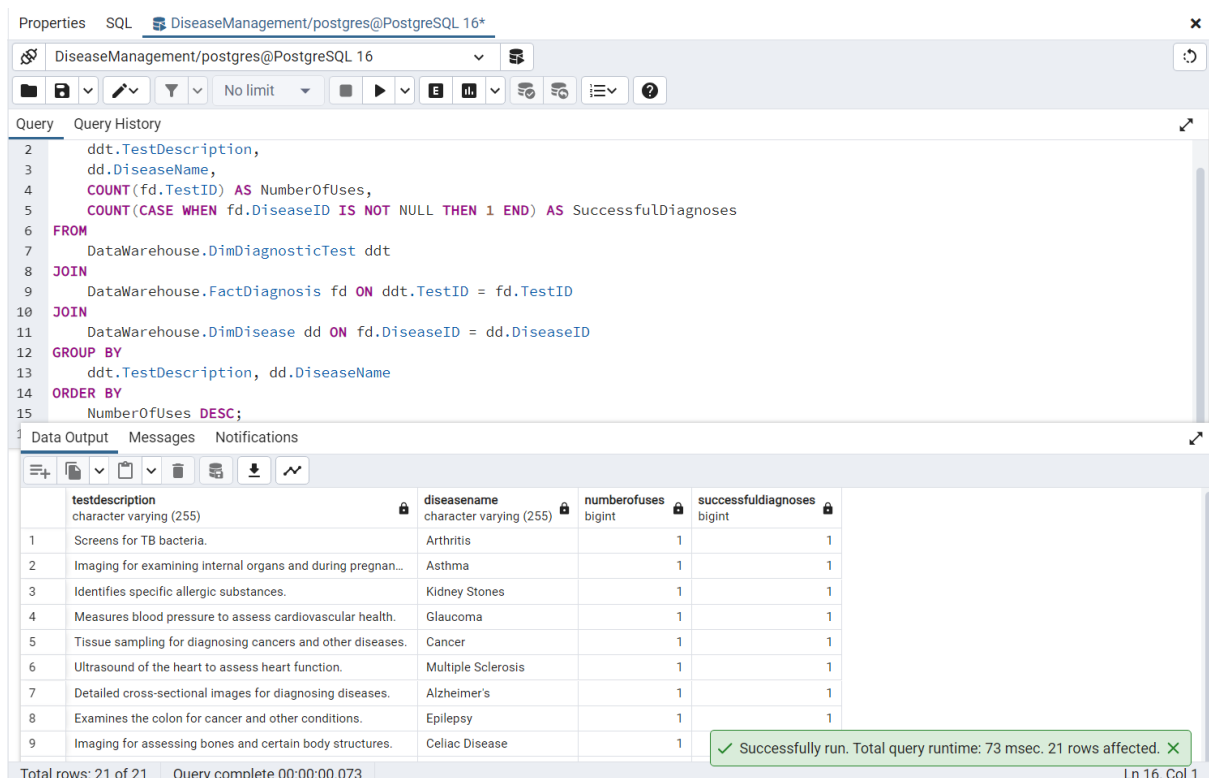
Successfully run. Total query runtime: 81 msec. 20 rows affected.

3) Diagnostic Test Usage and Diagnosis Correlation:

--Query:

```
SELECT
    ddt.TestDescription,
    dd.DiseaseName,
    COUNT(fd.TestID) AS NumberOfUses,
    COUNT(CASE WHEN fd.DiseaseID IS NOT NULL THEN 1 END) AS
SuccessfulDiagnoses
FROM
    DataWarehouse.DimDiagnosticTest ddt
JOIN
    DataWarehouse.FactDiagnosis fd ON ddt.TestID = fd.TestID
JOIN
    DataWarehouse.DimDisease dd ON fd.DiseaseID = dd.DiseaseID
GROUP BY
    ddt.TestDescription, dd.DiseaseName
ORDER BY
    NumberOfUses DESC;
```

Purpose: Analysing diagnostic test utilization and success rates in disease diagnosis could be helpful in test optimization and cost management.



The screenshot shows a database query tool interface. The top section displays the SQL query, and the bottom section shows the results of the query. The query is a SELECT statement that joins three tables: DimDiagnosticTest, FactDiagnosis, and DimDisease. It groups the results by TestDescription and DiseaseName, and orders them by the number of uses in descending order. The results table shows 9 rows of data, including test descriptions, disease names, and the number of uses and successful diagnoses for each.

testdescription	diseasename	numberofuses	successfuldiagnoses
Screens for TB bacteria.	Arthritis	1	1
Imaging for examining internal organs and during pregnan...	Asthma	1	1
Identifies specific allergic substances.	Kidney Stones	1	1
Measures blood pressure to assess cardiovascular health.	Glaucoma	1	1
Tissue sampling for diagnosing cancers and other diseases.	Cancer	1	1
Ultrasound of the heart to assess heart function.	Multiple Sclerosis	1	1
Detailed cross-sectional images for diagnosing diseases.	Alzheimer's	1	1
Examines the colon for cancer and other conditions.	Epilepsy	1	1
Imaging for assessing bones and certain body structures.	Celiac Disease	1	1

Successfully run. Total query runtime: 73 msec. 21 rows affected.

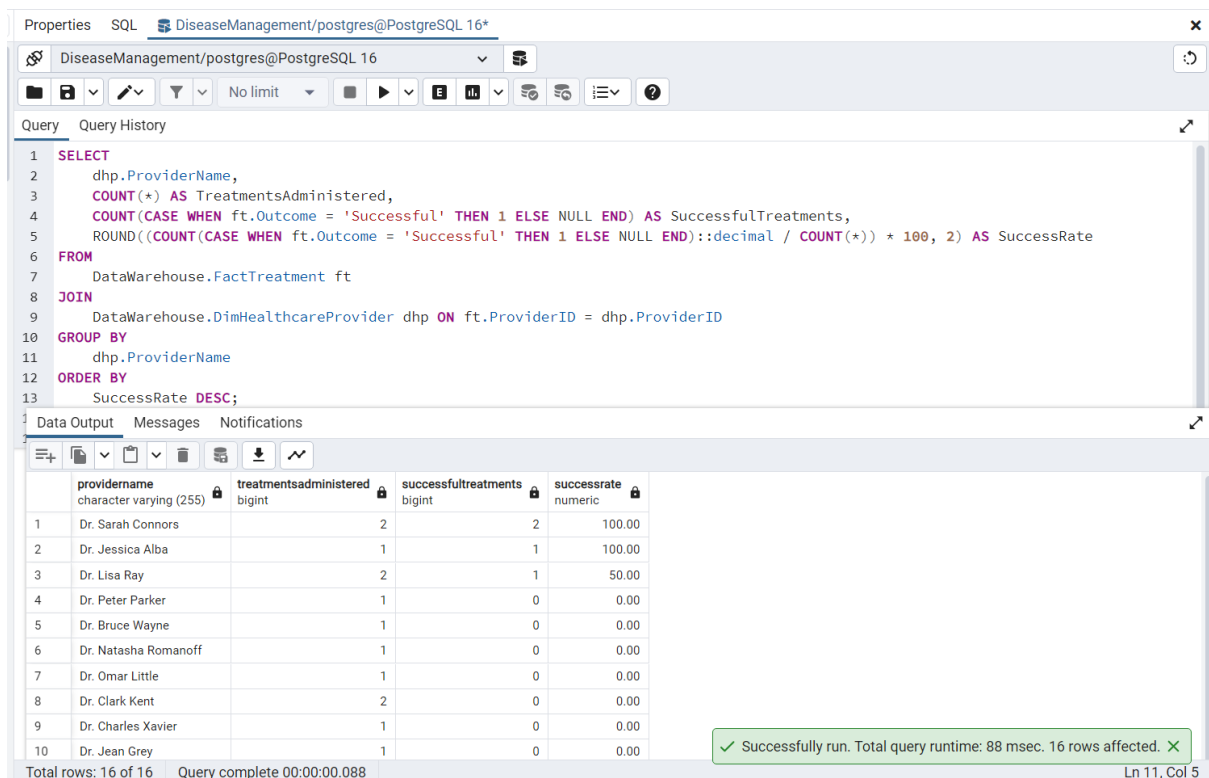
4) Healthcare Provider Efficiency :

--Query:

```
SELECT
    dhp.ProviderName,
    COUNT(*) AS TreatmentsAdministered,
    COUNT(CASE WHEN ft.Outcome = 'Successful' THEN 1 ELSE NULL END) AS
SuccessfulTreatments,
    ROUND((COUNT(CASE WHEN ft.Outcome = 'Successful' THEN 1 ELSE NULL
END)::decimal / COUNT(*)) * 100, 2) AS SuccessRate
FROM
    DataWarehouse.FactTreatment ft
JOIN
    DataWarehouse.DimHealthcareProvider dhp ON ft.ProviderID = dhp.ProviderID
GROUP BY
    dhp.ProviderName
ORDER BY
    SuccessRate DESC;
```

Purpose:

This query assists in determining which ones are the most successful by comparing the success rates of the treatments that various healthcare providers provide. And also assessing medical professionals according to how well their treatments work offers valuable information on staffing requirements and training.



The screenshot shows a PostgreSQL query editor interface. The query is executed, and the results are displayed in a table. The table has four columns: providername, treatmentsadministered, successfultreatments, and successrate. The results are sorted by successrate in descending order.

	providername	treatmentsadministered	successfultreatments	successrate
1	Dr. Sarah Connors	2	2	100.00
2	Dr. Jessica Alba	1	1	100.00
3	Dr. Lisa Ray	2	1	50.00
4	Dr. Peter Parker	1	0	0.00
5	Dr. Bruce Wayne	1	0	0.00
6	Dr. Natasha Romanoff	1	0	0.00
7	Dr. Omar Little	1	0	0.00
8	Dr. Clark Kent	2	0	0.00
9	Dr. Charles Xavier	1	0	0.00
10	Dr. Jean Grey	1	0	0.00

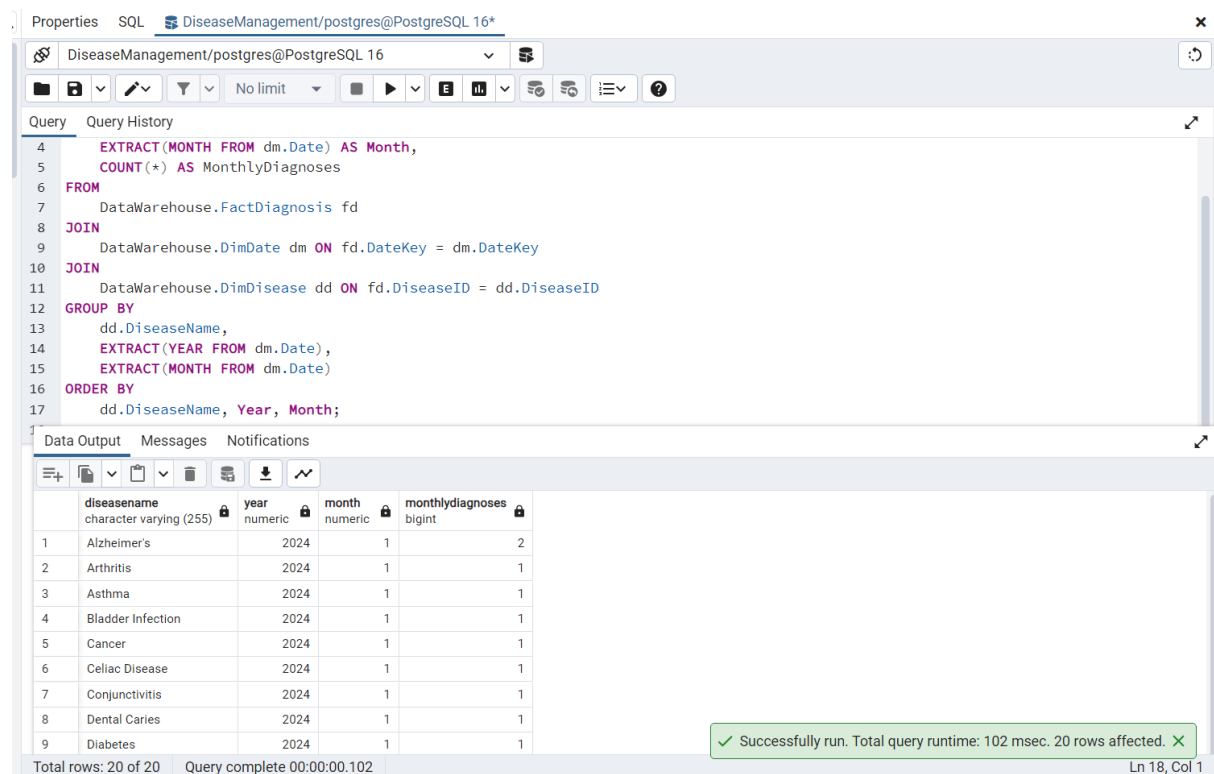
Successfully run. Total query runtime: 88 msec. 16 rows affected.

5) Trends in Disease Diagnosis Over Time:

--Query:

```
SELECT
    dd.DiseaseName,
    EXTRACT(YEAR FROM dm.Date) AS Year,
    EXTRACT(MONTH FROM dm.Date) AS Month,
    COUNT(*) AS MonthlyDiagnoses
FROM
    DataWarehouse.FactDiagnosis fd
JOIN
    DataWarehouse.DimDate dm ON fd.DateKey = dm.DateKey
JOIN
    DataWarehouse.DimDisease dd ON fd.DiseaseID = dd.DiseaseID
GROUP BY
    dd.DiseaseName,
    EXTRACT(YEAR FROM dm.Date),
    EXTRACT(MONTH FROM dm.Date)
ORDER BY
    dd.DiseaseName, Year, Month;
```

Purpose: By monitoring monthly diagnosis counts for every illness, this query enables medical professionals to observe changes in disease incidence over time and potentially link these trends to seasonal patterns or public health initiatives.



The screenshot shows a PostgreSQL query editor window titled "DiseaseManagement/postgres@PostgreSQL 16*". The query is displayed in the "Query" tab, and the results are shown in the "Data Output" tab. The query is a SQL statement that joins the FactDiagnosis, DimDate, and DimDisease tables to calculate the monthly diagnosis counts for various diseases. The results table has four columns: diseaseName, year, month, and monthlydiagnoses. The data shows 20 rows of results, with the first 9 rows visible in the screenshot. The diseases listed are Alzheimer's, Arthritis, Asthma, Bladder Infection, Cancer, Celiac Disease, Conjunctivitis, Dental Caries, and Diabetes. All diagnoses occurred in the year 2024. The number of diagnoses per month is 2 for Alzheimer's and 1 for all other diseases.

diseaseName	year	month	monthlydiagnoses
Alzheimer's	2024	1	2
Arthritis	2024	1	1
Asthma	2024	1	1
Bladder Infection	2024	1	1
Cancer	2024	1	1
Celiac Disease	2024	1	1
Conjunctivitis	2024	1	1
Dental Caries	2024	1	1
Diabetes	2024	1	1

Successfully run. Total query runtime: 102 msec. 20 rows affected.

8. Utilizing the Queries:

All those queries benefit various healthcare system stakeholders:

- Healthcare administrators: They could better manage resources and develop training programs by using information from treatment outcomes and provider efficiency.
- Doctors and Medical Staff: Gain an understanding of which treatments are most effective for which diseases and adapt their practices accordingly.
- Public health officials: They can use illness trend data to gauge the effectiveness of public health initiatives or to develop responses to disease outbreaks.
- Research and Development Teams: Analyze diagnostic test usage and success rates in order to improve or create new diagnostic instruments.

9) No SQL and Relational Contrast:

Understanding the main distinctions between these database systems and how they impact data modelling, querying, and storage is crucial when thinking about converting a relational database structure to a NoSQL setting. Let's use Neo4J, a graph-based NoSQL database, and MongoDB, a document-oriented NoSQL database, as our NoSQL alternatives for the purposes of illustration and clarity.

Relational Database Structure: The structure is highly normalized in a typical relational database design, such as the one we have for a healthcare data warehouse, to minimize redundancy and guarantee data integrity.

Relationships: Foreign keys are used to link tables. For example, the Patient table might have foreign keys linking to the Diagnostic Test table.

Data integrity: Data integrity is maintained by normalization rules and foreign keys, which make sure that there is as little duplication of data as possible. NoSQL databases frequently give precedence to availability and partition tolerance (AP of the CAP(Consistency, Availability, Partition Tolerance) theorem), with eventual consistency, whereas relational databases impose the ACID qualities (Atomicity, Consistency, Isolation, Durability). This trade-off can influence judgments in situations where quick consistency is essential.

Query Complexity: SQL queries are capable of handling complex questions, particularly ones that call for numerous joins and aggregations. Unless the data is designed especially to fit the queries, NoSQL databases may handle them less well, which frequently results in data duplication or complicated nested queries in MongoDB or traversal operations in Neo4J's.

Transferring data from a relational database to a NoSQL database requires reconsidering the data relationships, access patterns, and integrity constraints, in addition to simply translating tables to documents or nodes. Whether it's MongoDB's scalability and flexibility or Neo4J's

intricate relationship mapping, the database type selection should be based on the particular requirements of the application.

Transition to MongoDB (Document-Oriented NoSQL):

Document Structure: MongoDB stores data in documents that are formatted in BSON format, which is comparable to JSON, as opposed to relational databases, which employ tables, rows, and columns. Compared to the flat relational model, this format allows for a more deep and flexible structure because it can hold arrays and subdocuments and naturally supports hierarchical relationships.

Flexible Schema: Unlike relational databases, MongoDB does not require a pre-established schema in order to insert data. This implies that there may be variations in the structure of each document. This means that without changing the database structure, each patient record in the healthcare data warehouse can, for example, have distinct properties relevant to particular diagnoses or treatments.

Denormalization: While relational databases typically normalize data to avoid redundancy, MongoDB benefits from a certain level of denormalization as well. For example, to minimize the amount of reads and joins, you may embed the disease and treatment data directly into each patient record rather than having separate tables for the patient, disease, and treatment categories.

Benefits: Highly flexible and scalable, especially for applications with varied and evolving data models. Ideal for handling large volumes of data with minimal relationships.

Transition to Neo4J (Graph-Based NoSQL):

Nodes and Relationships: In Neo4J, data is stored in nodes (which can be thought of as records) connected by relationships. Properties may be attached to any node or connection. Compared to relational databases, which employ foreign keys to represent relationships, this paradigm is very different. Relationships could be used to connect Patients, Diseases, Treatments, and Healthcare Providers as nodes in the healthcare scenario.

Data Retrieval: Relationships between data points are often just as significant as the data itself, which is where graph databases shine. For example, it is possible to simplify complex queries that would otherwise need substantial joins in a relational system by immediately comprehending how patients are connected to healthcare providers and their conditions through direct relationships.

Suitability for Use Cases: Especially useful for examining relationships and trends, such as the transmission of infectious diseases within a community or the effectiveness of referral networks in medical facilities.

Conclusion:

Specific application needs should choose which of these database types to use: relational for tight consistency and structured data, MongoDB for flexible schema and quick scaling, and Neo4J for complex relationship data. Each offers advantages, and the decision we make will mostly rely on the particular needs and difficulties your healthcare data management system

presents. The essential distinctions between the data structures and their effects on application performance and reliability should be emphasized in this description.

10) Architecture and Process (AWS Software and products)

AWS Architecture for Data Processing:

The objective is to build a versatile and scalable architecture on AWS that can easily manage batch and real-time data processing, guaranteeing that data is available for analysis almost immediately after it is generated, effectively using Lambda Architecture principles.

The architecture includes Amazon S3, Amazon Kinesis, Amazon EMR, Amazon Redshift, and, optionally, Amazon DynamoDB. These components cover data ingestion, storage, processing, and querying, all of which are essential to the Lambda Architecture.

Batch Data Loading: The data lake, Amazon S3, receives data uploads. This data is periodically processed in batch mode by Amazon EMR, which loads the processed data into Amazon Redshift for querying after completing transformations and analytics.

Real-Time Data Loading: Spark Streaming on Amazon EMR instantly processes real-time data streams received through Amazon Kinesis, minimizing processing delay. After processing, the data can be put into Amazon Redshift for integrated analytics or saved in Amazon DynamoDB for easy access.

Key Components:

Amazon S3 : This service is used to store all raw data. acts as the primary data lake where all incoming data whether it comes in streams or batches, is initially landed.

Amazon Kinesis : It is a service for managing real-time data streams. It instantly feeds real-time data into the processing systems by capturing it from multiple sources.

Amazon EMR : It provides Spark hosting for batch and real-time processing. EMR periodically executes jobs to process historical data stored in S3 for batch processing. It handles data arriving from Amazon Kinesis using Spark Streaming for real-time processing, processing it as it comes in.

Amazon Redshift : It serves as the serving layer, loading processed data from both batch and real-time processes. Large datasets can be queried complexly using this robust data warehouse, which integrates the output of both processing methods to provide thorough insights.

Amazon DynamoDB : Optionally used for storing outputs from real-time processing, enabling quick access to the most recent data.

Simplified Data Flow :

Utilization:

(1) Batch Data: Uploaded straight to Amazon S3.

(2) Real-Time Data: Streamed via Amazon Kinesis.

(3) Processing:

(i) Batch Processing: S3 data is processed using scheduled Amazon EMR jobs, which also include complex computations and format conversions.

(ii) Real-Time Processing: Incoming streams from Kinesis are processed instantly by Amazon EMR with Spark Streaming, resulting in the least amount of delay possible.

(4) Data Serving and Storage:

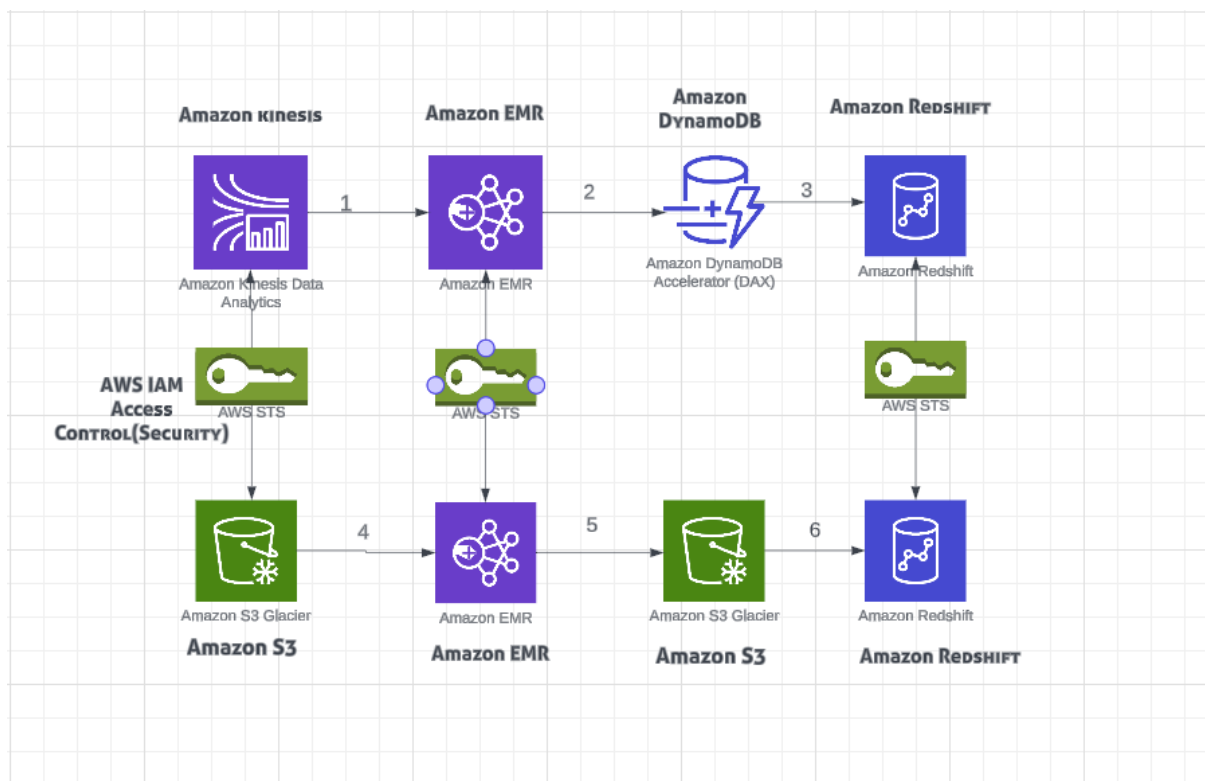
(i) Processed Batch Data: Loaded into Amazon Redshift for querying and stored back in S3.

(ii) Processed Real-Time Data: Instantaneously available in Amazon DynamoDB and combined with Redshift for unified querying.

AWS's Lambda Architecture for Handling Batch and Real-Time Data:

- Amazon Elastic MapReduce (Amazon EMR) is the AWS service typically used for processing large volumes of semi-structured or unstructured data.
- A Lambda Architecture system, integrating Amazon Kinesis, Amazon Simple Storage Service (Amazon S3), Spark Streaming, and Spark SQL on an Amazon EMR cluster, enables real-time data stream processing.
- This Lambda Architecture approach merges batch processing with stream (real-time) data processing.

Architecture Diagram:



Explanation:

Access Control/Security (Amazon IAM): Implements security guidelines for all system communications and data flows. It ensures that only authorized individuals and systems can access or utilize your AWS resources by managing permission and authentication for all involved AWS services.

(1)Amazon Kinesis to Amazon EMR: Real-time data is captured by Amazon Kinesis and sent to Amazon EMR for immediate processing.

(2)Amazon EMR to Amazon DynamoDB: Processed real-time data is stored in Amazon DynamoDB for quick access and retrieval.

(3)Amazon DynamoDB to Amazon Redshift: Data from DynamoDB is loaded into Amazon Redshift for integrated querying and analysis using Spark SQL.

(4)Amazon S3 to Amazon EMR: Raw batch data is taken from Amazon S3 and processed by Amazon EMR.

(5)Amazon EMR to Amazon S3: Processed batch data is stored back into Amazon S3 for persistence and further use.

(6)Amazon S3 to Amazon Redshift: Processed batch data from Amazon S3 is loaded into Amazon Redshift for comprehensive data analysis.

All the stages guarantee that data flows smoothly from storage to processing to ingestion, which in turn makes integrated analytics on Amazon Redshift.

Security, High Performance, and Resilience:

Resilience:

- High availability and durability are ensured by utilizing several AWS services, such as Amazon S3, DynamoDB, and Redshift, which by default offer data redundancy and redundancy capabilities.
- Replication of data across many EC2 instances in different Availability Zones is a feature of Amazon EMR that improves processing layer resilience.

High Performance:

- To handle tasks effectively, Amazon EMR offers a scalable environment. For huge datasets, Amazon Redshift provides quick query performance.
- Real-time data processing is ensured using Amazon Kinesis. A high-performance solution is guaranteed when these services are combined.

Security:

- AWS comes with several built-in security features, including data encryption for data in transit and at rest (in S3, DynamoDB, and Redshift), IAM for fine-grained access control, and VPCs for resource isolation and network access management.
- Together, these elements contribute to the application's and data's security.

Using “Lambda Architecture principles” for batch and real-time data processing, the offered solution fits the needs of establishing a secure, high-performing, and robust application on AWS. It covers all aspects of the architecture configuration, including the components used, data processing techniques, and crucial non-functional needs like security and resilience.

How Snowflake Enhances Our Model:

Snowflake is a cloud-native data warehousing service that excels in scalability, performance, and ease of maintenance, making it an appealing alternative to traditional database systems such as PostgreSQL for specific use cases, particularly in data warehousing and big data analytics.

Here are some specific advantages utilizing Snowflake could have for our application:

Efficiency and Performance when Processing JSON

Efficient JSON Querying:

- Organizations can effortlessly carry out complicated data analysis directly on semi-structured data because to Snowflake's native support for JSON and other semi-structured data formats and its effective querying capabilities.
- This simplifies the data analysis process by doing away with the requirement for significant preprocessing or transformation that PostgreSQL could demand.

Elasticity and Scalability

Consistent Scaling Without Interruptions:

- The architecture of Snowflake allows for the unique capability of dynamically scaling compute resources without interfering with current processes.
- This means that Snowflake may dynamically deploy extra computing resources to maintain performance during periods of unexpected demand or intense querying, a process that can be more complex and less fluid with PostgreSQL.

Efficiency in Maintenance and Operations

Reduction of Infrastructure Management:

- Snowflake helps data teams to focus more on providing value through data analytics by abstracting the complexity of infrastructure administration, freeing up time that would otherwise be spent on maintenance duties.
- When compared to PostgreSQL, where database tuning, updates, and scalability might require substantial resource consumption, this service approach is particularly advantageous.

Data Engineering Resource Optimization:

- By reducing the amount of time data engineers spend on repetitive data management chores, Snowflake's streamlined management allows them to concentrate on more strategic data efforts.
- This change has the potential to improve data teams' overall creativity and productivity.

Data recovery and reliability

Enterprise-Level SQL Reliability and Support:

- Snowflake's strong ANSI SQL support guarantees compatible with both legacy SQL-based tools and business apps, making integration simpler.
- Reliability when managing intricate data operations is improved by its capability for multi-statement transactions.

Advanced Data Retention and Recovery:

- Snowflake protects data from loss resulting from operational mistakes, hardware malfunctions, and other unanticipated events by providing automatic data replication and failover capabilities.
- Effective point-in-time recovery is made possible by the platform's capacity to store data changes for up to 90 days, which is essential for preserving data integrity and business continuity.

